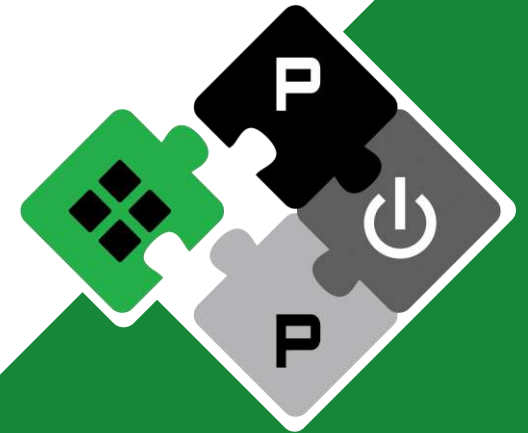


FlooNoC: A Multi-Tbps Wide NoC for Heterogeneous AXI4 Traffic

Integrated Systems Laboratory (ETH Zürich)

Tim Fischer

fischeti@iis.ee.ethz.ch



PULP Platform

Open Source Hardware, the way it should be!

@pulp_platform 

pulp-platform.org 

youtube.com/pulp_platform 

The heart of an HPC system is a compute cluster



Compute cores

- 8 RISC-V compute cores
- Tightly coupled accelerator (e.g. FPU)
- Private L0 & Shared L1 I\$

Local Scratchpad Memory

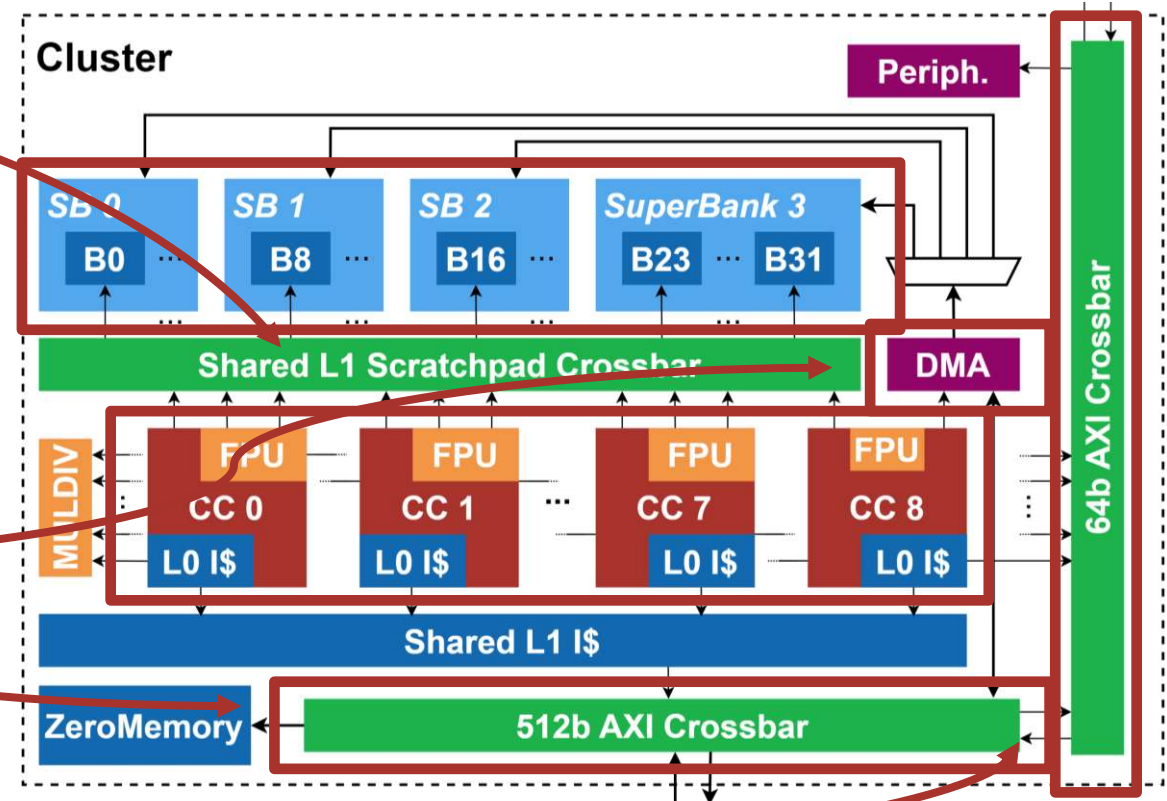
- Low-Latency interconnect
- Multi-banked

DMA

- Keep cores busy with new data
- Wide 512-bit AXI Bus
 - Burst transfers, Multiple Outstanding Transactions

Control & Synchronization bus

- Narrow 64-bit AXI Bus



Then we can scale out

6 Groups: 216 cores/die

- 4 cluster / group:
- 8 compute cores / cluster
- 1 DMA core / cluster

High Bandwidth Memory (HBM)

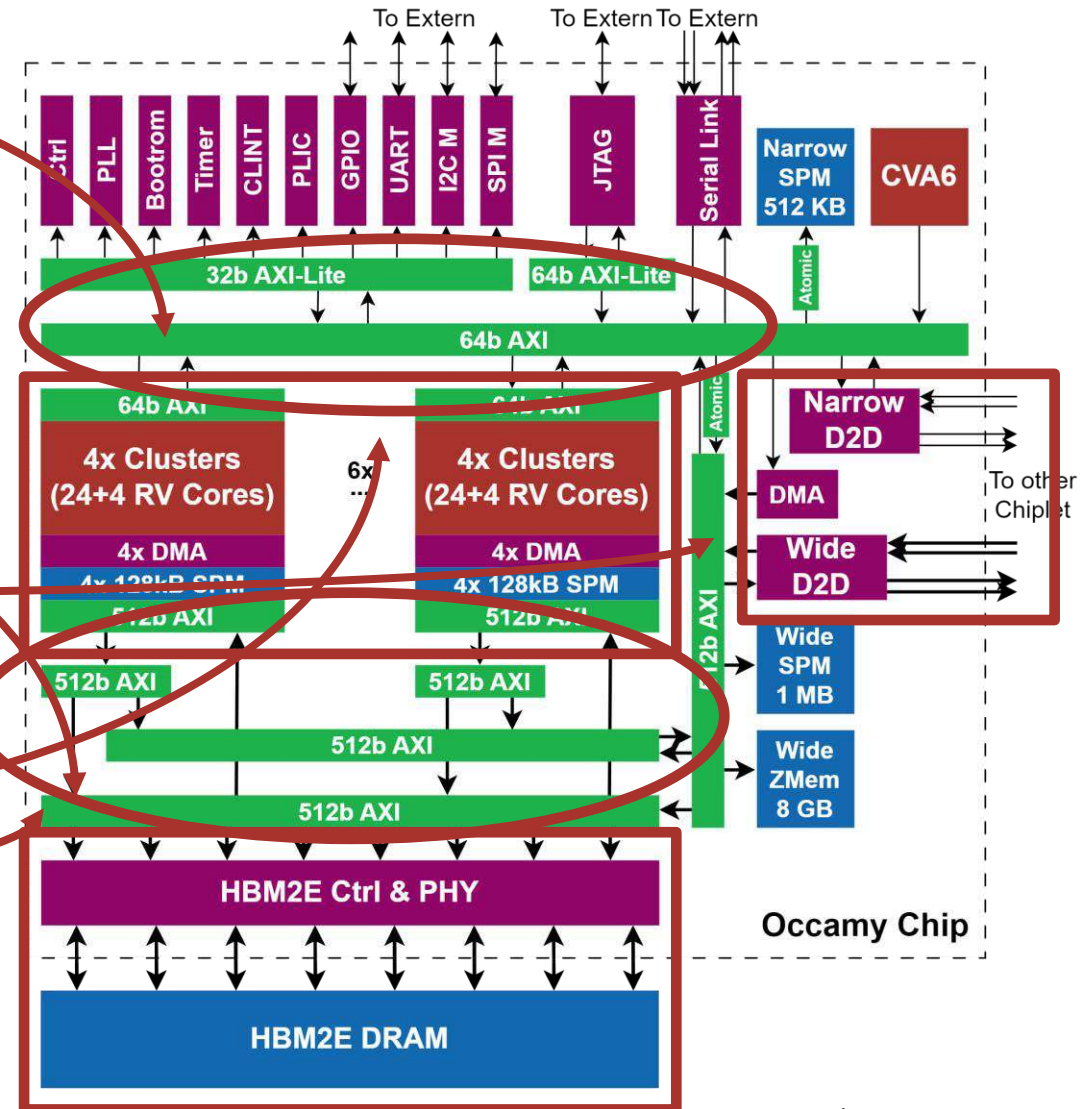
- 8x 512-bit channels

Off-chip D2D Link

- For scaling out to chiplets

Interconnect

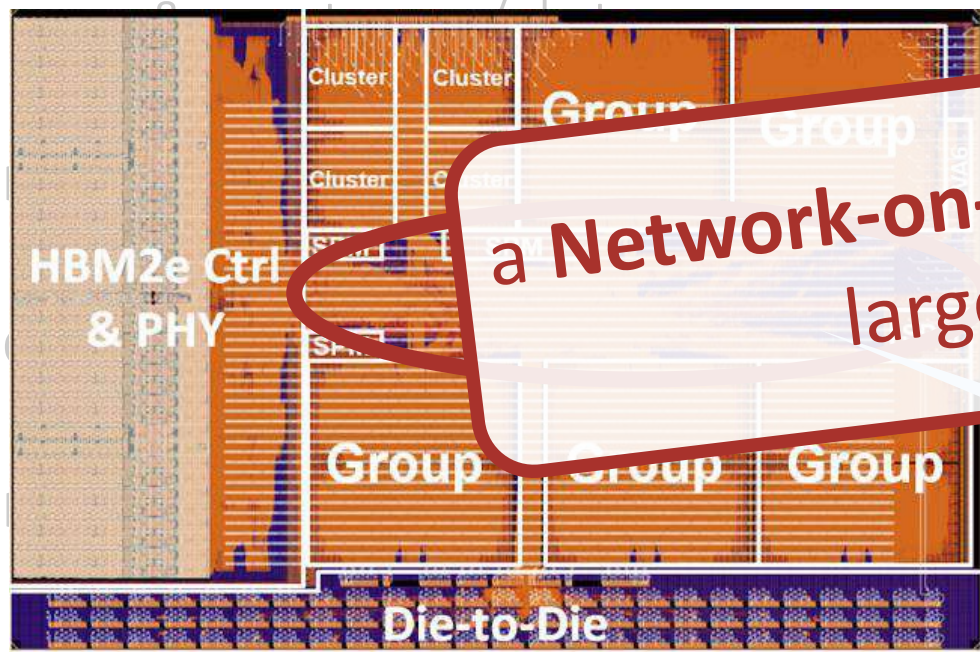
- Based on AXI
- 64-bit Narrow Control Plane
- 512-bit Wide Data Plane



This is Occamy, a real chip we taped-out

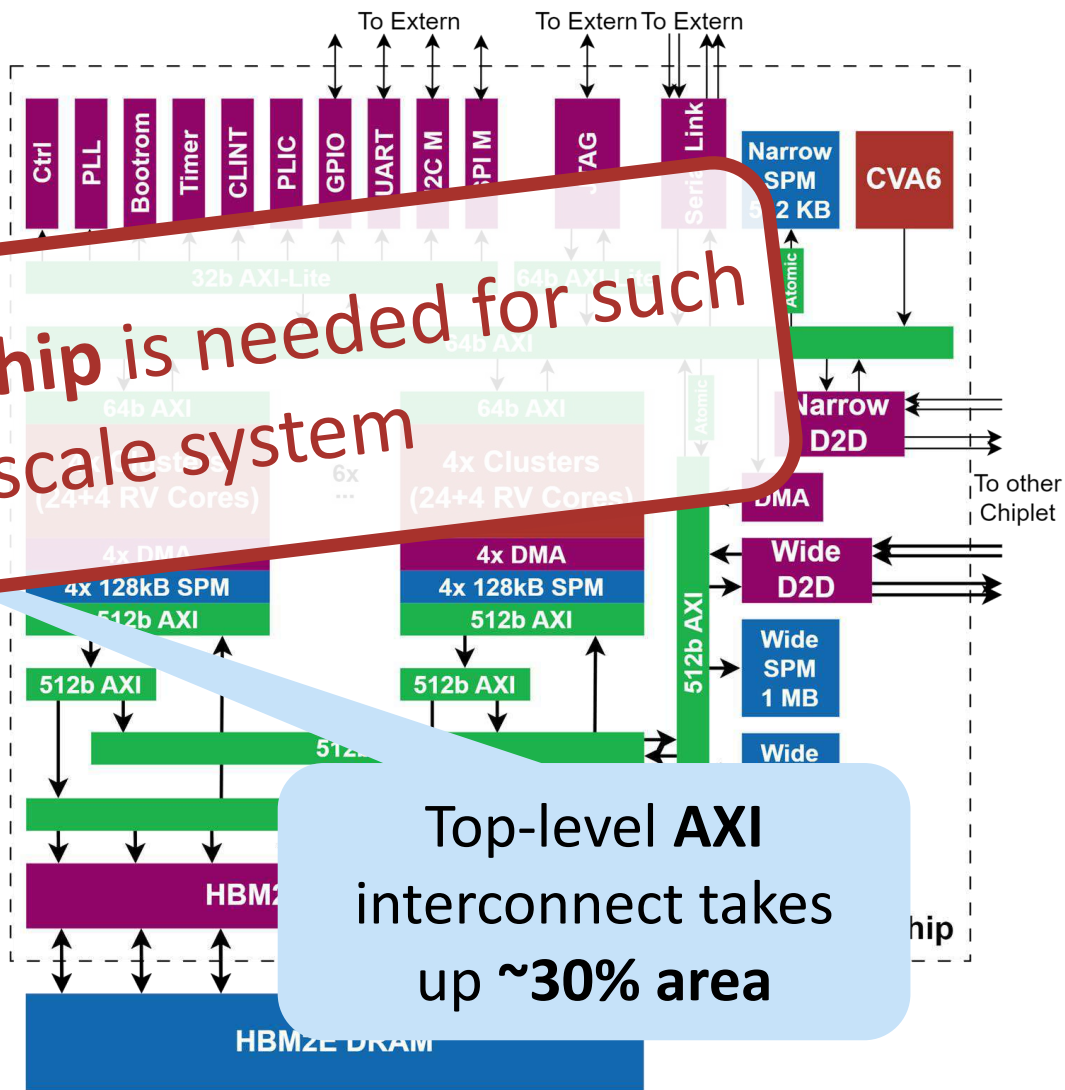
6 Groups: 216 cores/die

- 4 cluster / group:



- 512-bit Wide Data Plane

a Network-on-Chip is needed for such large-scale system



Top-level AXI interconnect takes up ~30% area



How to design a NoC for the modern age?



How do we overcome the **memory wall**?

- Many applications today are **memory-bound** e.g. LLM, ChatGPT
- Access to DRAM is expensive, **100x cycles** of latency
- The interconnect needs to be both **high-bandwidth** and **latency-tolerant**

How do we handle **heterogeneous** traffic?

- **Burst-based** data transfers for large data movements
- **Latency-sensitive** messages for synchronization

How do we make it **scale**?

- The area should be **dominated by compute logic**
- **Spend energy on computation**, not data movement

Our Contributions



First open-source fully AXI4 compliant NoC

- Open-source (closed-source solution ARM CoreLink, Arteris FlexNoC)
- Full support for multiple outstanding transactions and reordering
 - Limited or no support in (ESP, constellation, OpenPiton)

Support for differentiated traffic classes

Strong physical awareness of the design

- Exploiting the degree of freedom given by on-chip wires
- Very large physical channels and routing over the tile

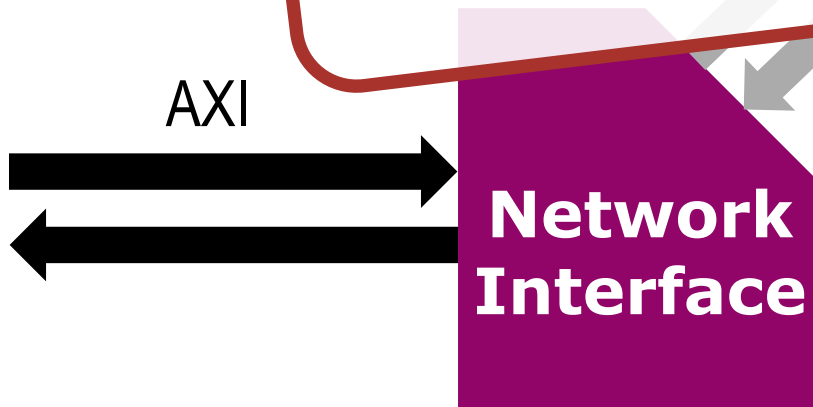
Detailed physical implementation in an advanced technology

Decouple AXI with link-level protocol



Network Interface

- Protocol conversion from AXI to link-level
- Supports **AXI features**
 - Burst transfers
 - Outstanding Transactions
 - Atomics
- **Reorder Buffer** to guarantee ordering



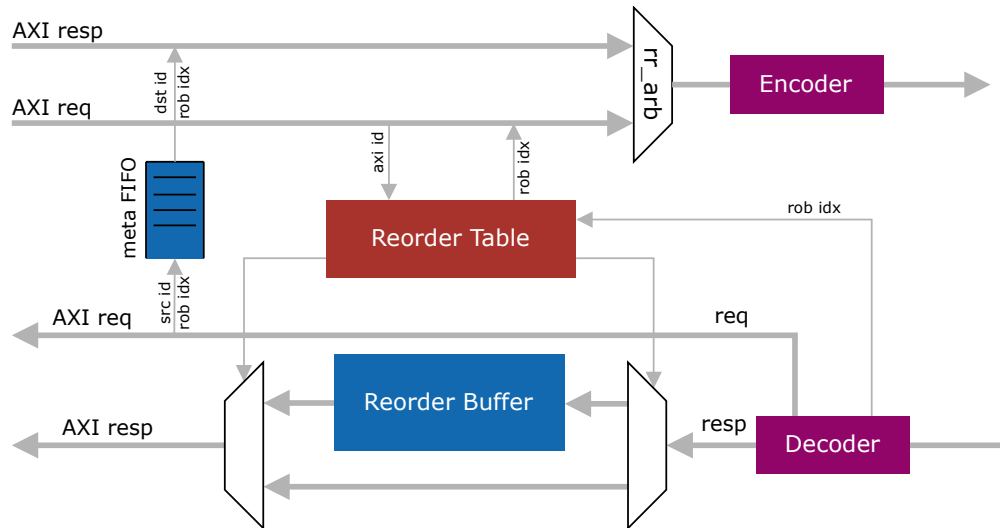
Move complexity to the entry point & amortize cost with **low-complexity** routers

- **Simple & Streamlined**
- **State-less**
 - Fire & Forget
 - No tracking of packets
- **Better scalability**
 - multi-hop networks

Managing the ordering requirement of AXI



Transactions with the **same ID**
must be **ordered** in AXI



- **Reorder Table**

- Keeps track of outstanding transactions of each AXI ID
- Keeps state of the order of responses

- **Reorder Buffer**

- Dynamically allocated
- Temporarily stores AXI responses that are out of order

- **Optimized for deterministic routing**

- First/single response is always in order
- Order is guaranteed for the same destination

Routers can be streamlined

Input Buffers

- Configurable Depth

Output Buffers (Optional)

- Easier Timing closure

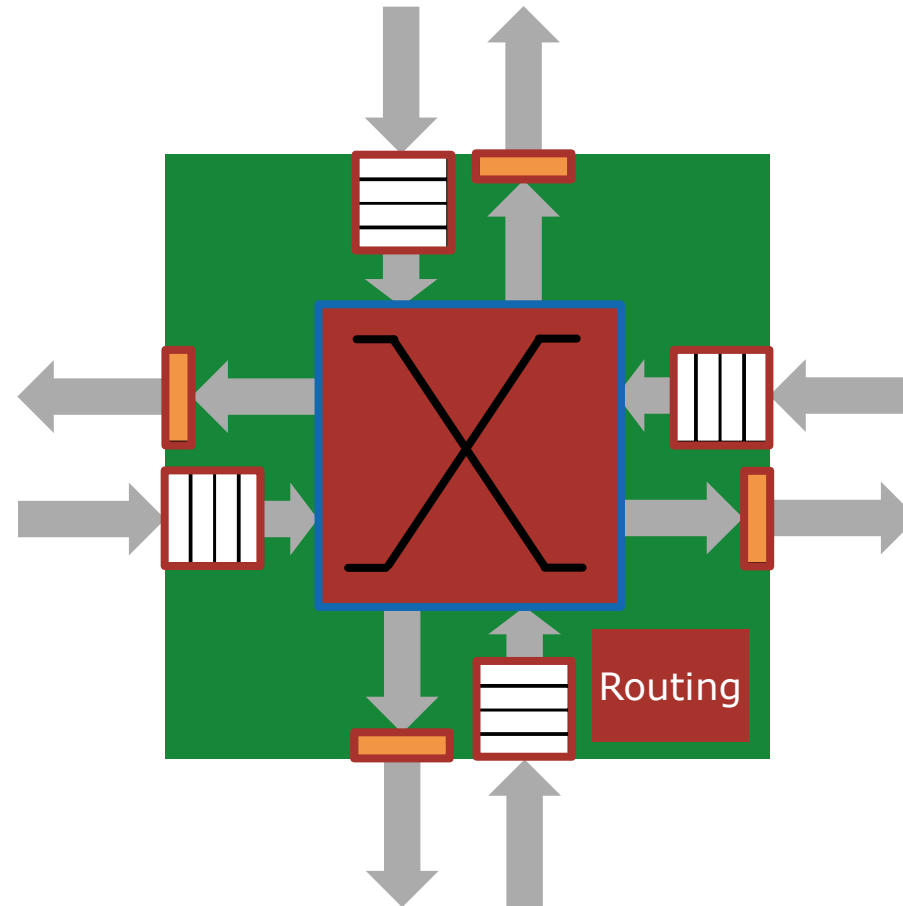
1-cycle latency

- +1 for output buffers

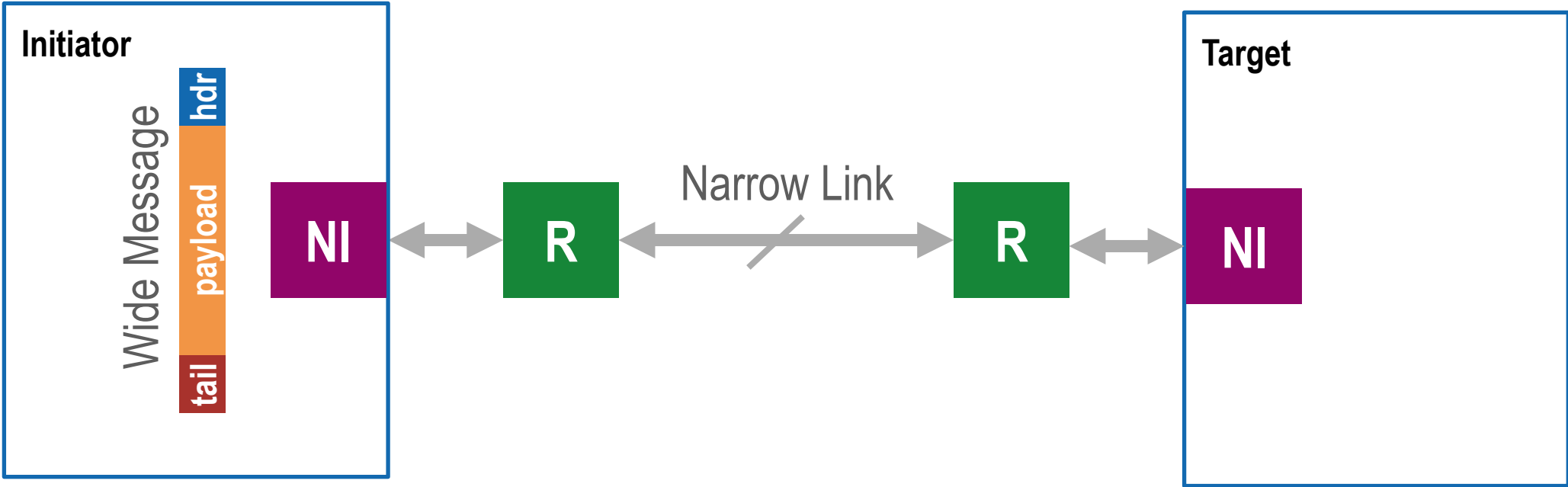
Crossbar Switch

- Round-Robin Arbitration
- Wormhole routing

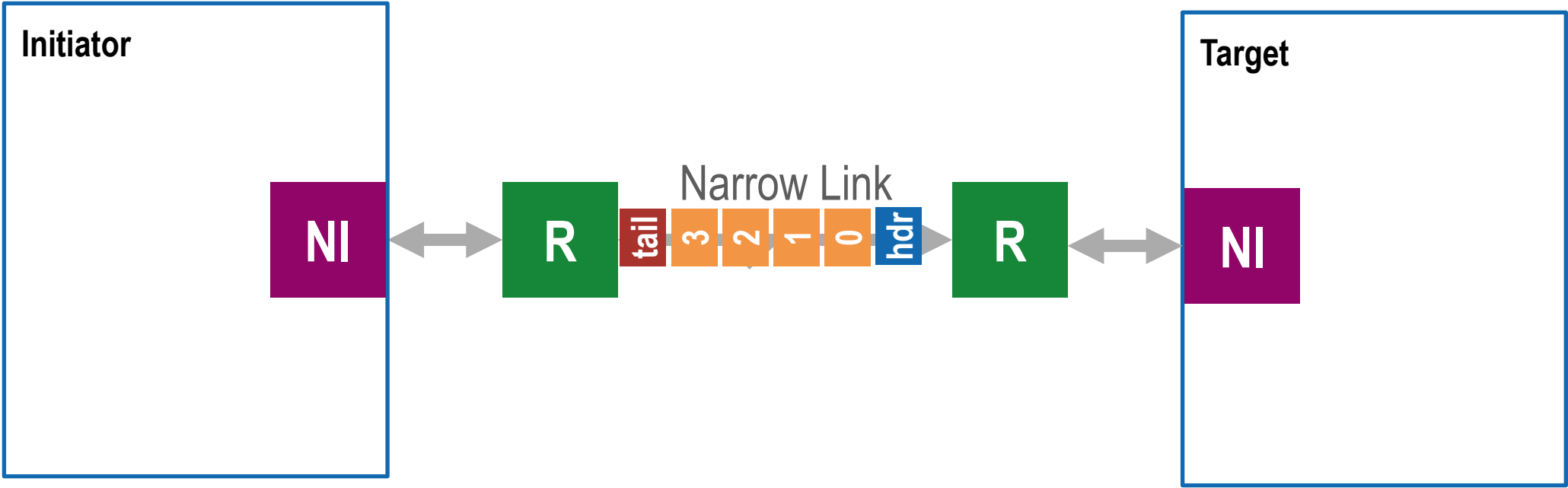
Separate Routing module



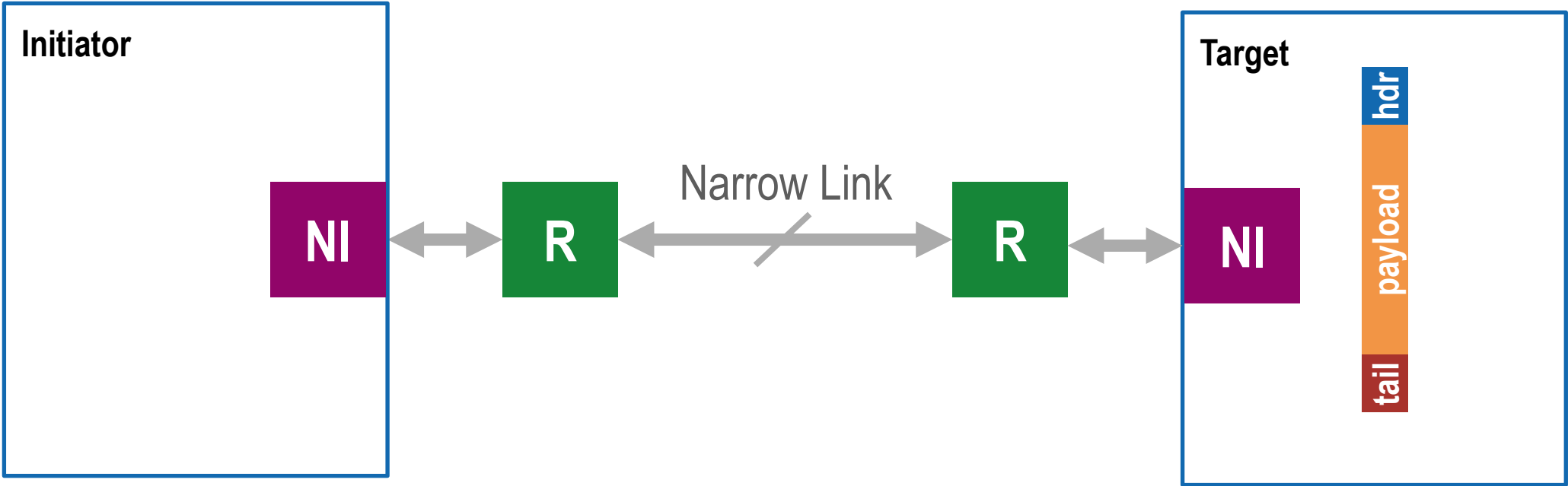
The bandwidth problem with serialization



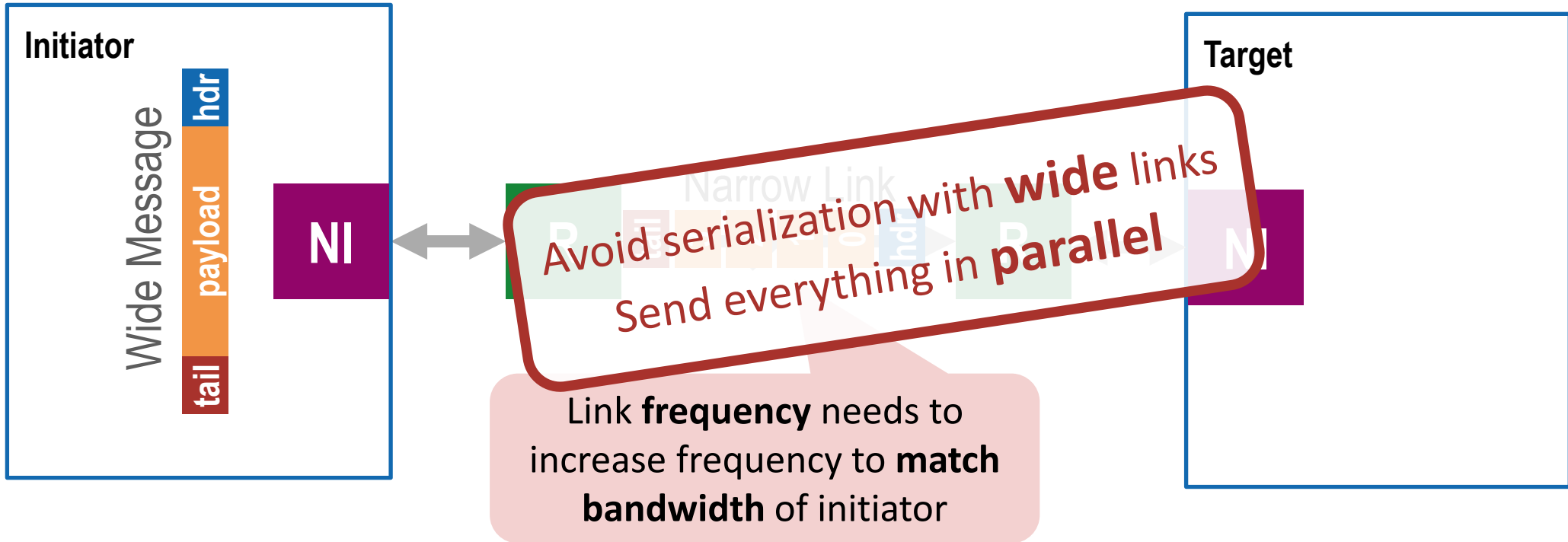
The bandwidth problem with serialization



The bandwidth problem with serialization



The bandwidth problem with serialization



Not possible anymore if bandwidth injection is too high

Physical vs. Virtual Channels



Virtual Channels

Time multiplexing onto physical channel(s)

Arbitration complexity

Bandwidth reduction

Fewer wires, less routing

Physical Channels

Separate networks for all channels

Physical channels are the way to go in modern Network-on-Chips

More wires, more routing

Routing resources are **plentiful** in modern technologies

e.g. GF12, 13M → ~50k wires/mm

How to handle heterogeneous traffic



High Bandwidth traffic

- DMA traffic
- Wide busses (512-bit)
- Burst-based
- Large messages (few kBs)
- Latency tolerant

Latency-sensitive traffic

- Core traffic
- Atomics
- Synchronization
- Single-word messages

Auxiliary traffic

- Request messages
- Acknowledgment messages (AXI write responses)
- Small messages

Wide links
with **high utilization**

Narrow links with
low utilization

Wide

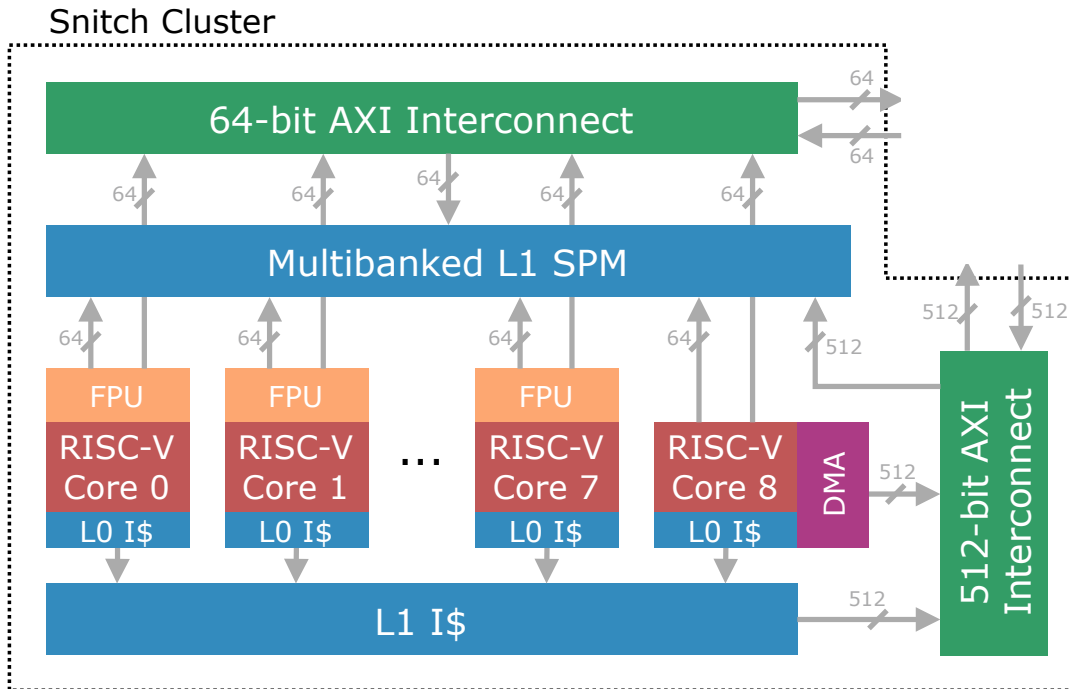
Req

Rsp

Case Study: Snitch Cluster



- **Snitch Cluster**
 - 9 RISC-V + DMA
 - **128KB** TCDM
 - **Two** bidir AXI interfaces
 - **Narrow** 64-bit
 - **Wide** 512-bit

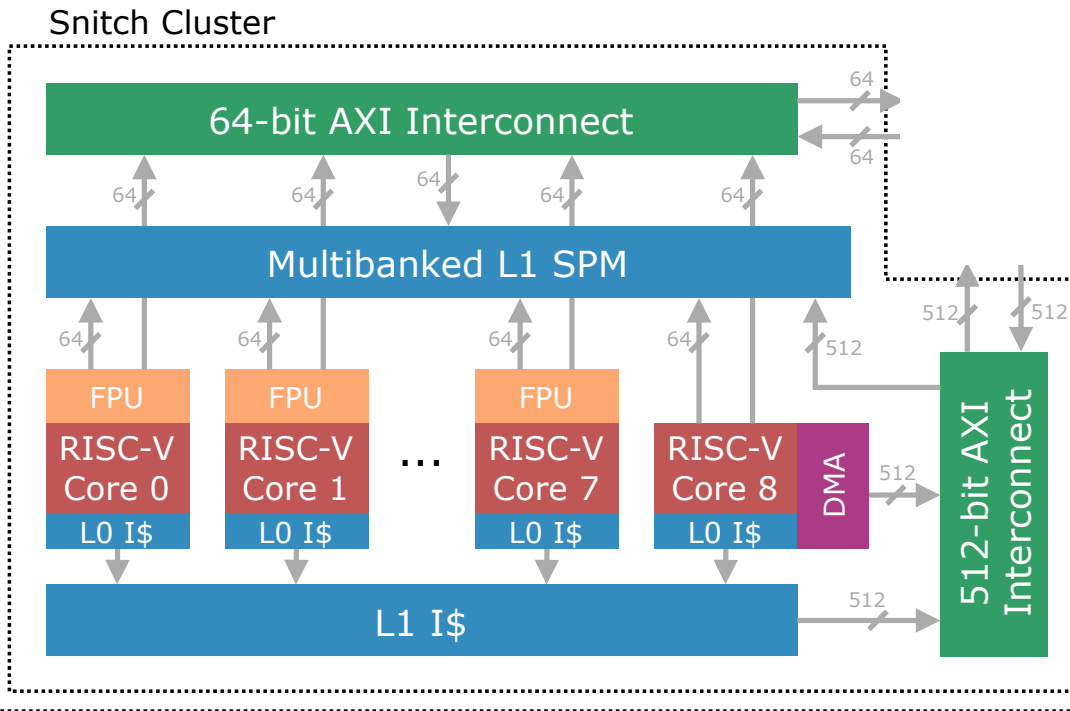


Case Study: Snitch Cluster



Compute Tile

⇒ NoC Links → AXI4 Bus



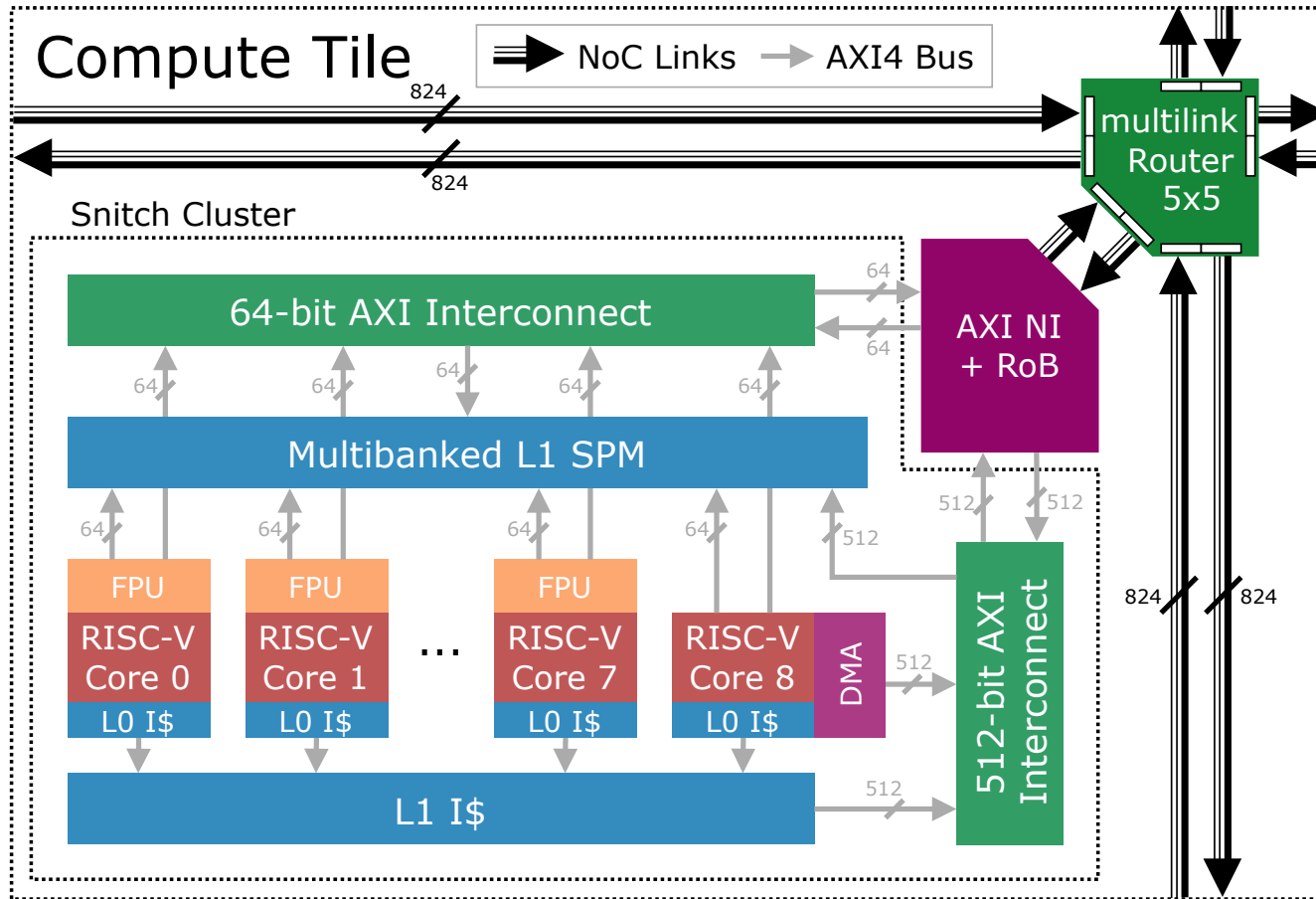
- **Snitch Cluster**

- 9 RISC-V + DMA
- **128KB** TCDM
- **Two** bidir AXI interfaces
 - **Narrow** 64-bit
 - **Wide** 512-bit

- **Compute Tile**

- Wraps a Snitch Cluster
- Network Interface with **8KB** Reorder Buffer
- 5x5 Multi-link Router

Case Study: Snitch Cluster



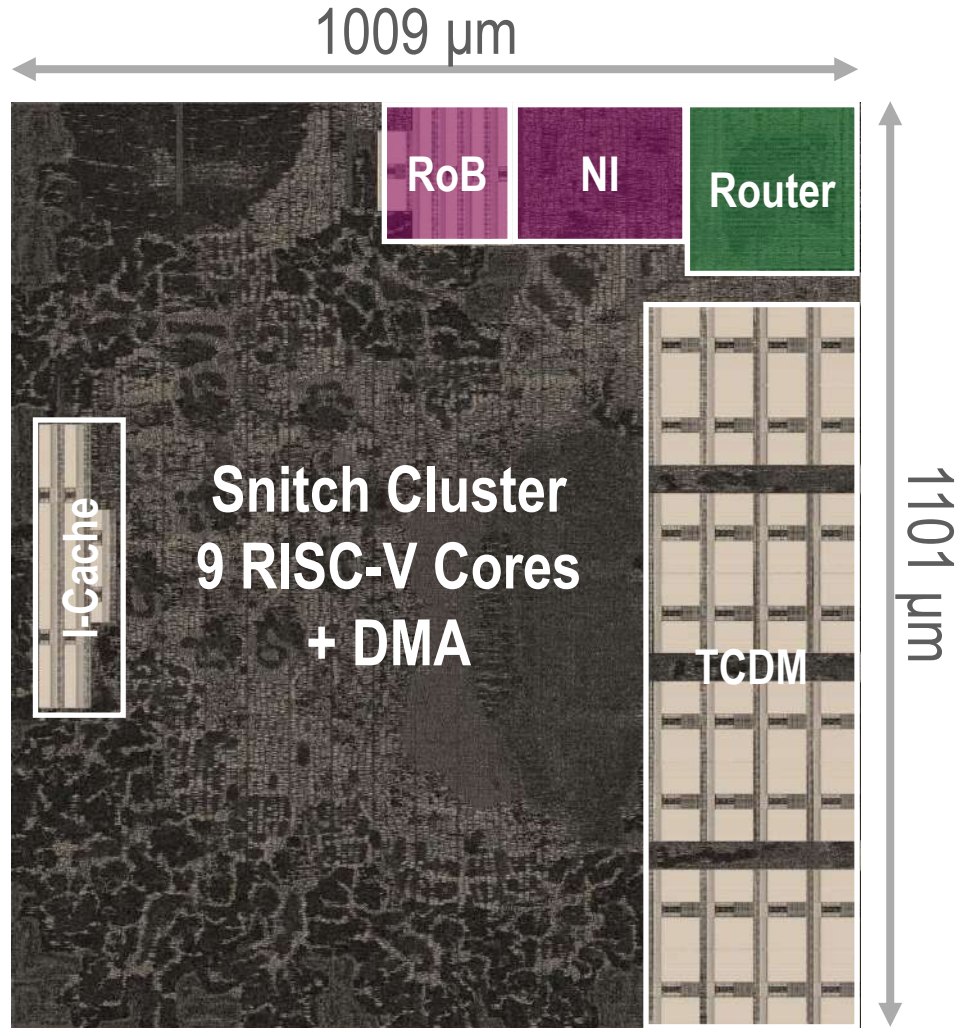
• Occamy Snitch Cluster

- 9 RISC-V + DMA
- **128KB** TCDM
- **Two** bidir AXI interfaces
 - **Narrow** 64-bit
 - **Wide** 512-bit

• Compute Tile

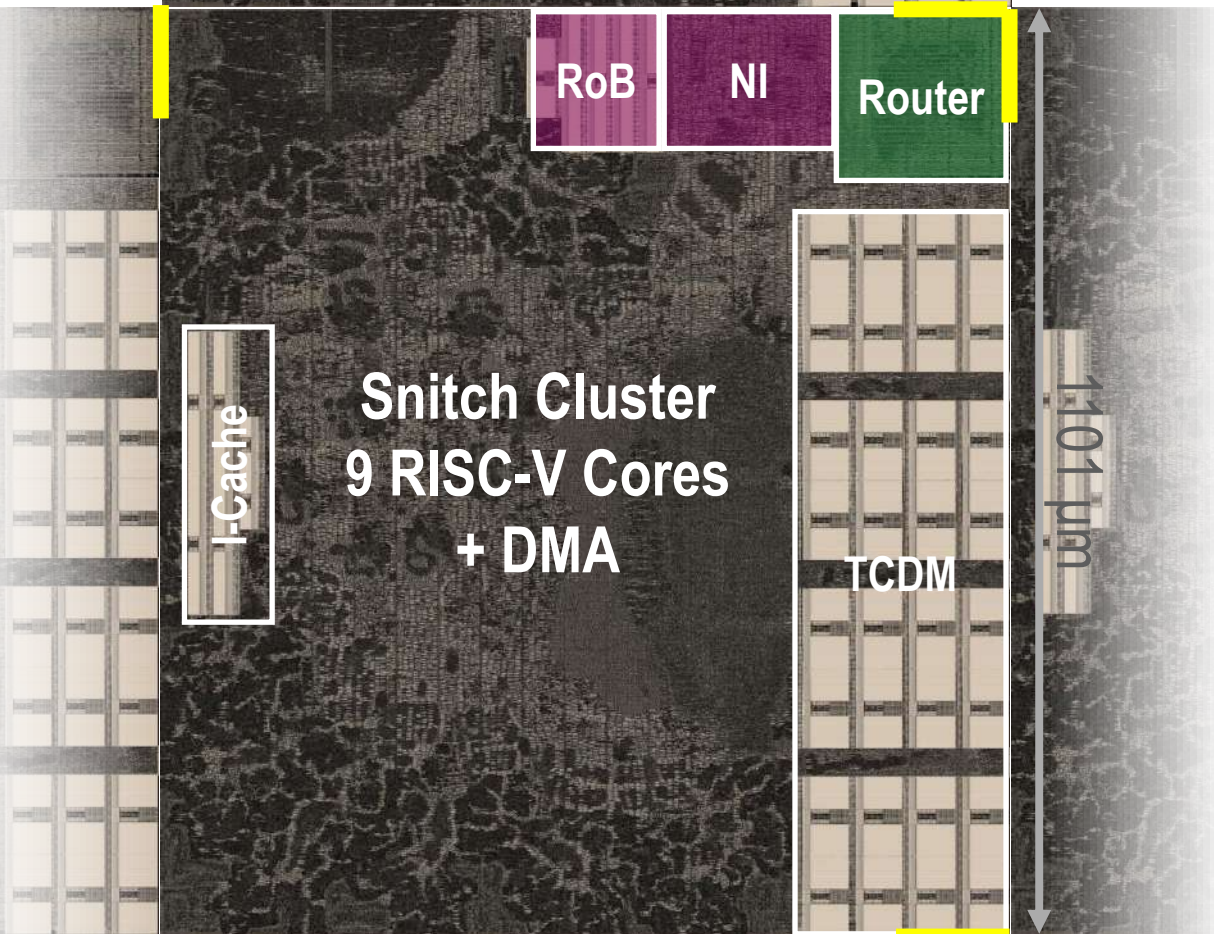
- Wraps a Snitch Cluster
- Network Interface with **8KB** Reorder Buffer
- 5x5 Multi-link Router
- Can be arranged in a mesh

Case Study: is physical implementation feasible?



- Hard Router Macro
- Flattened Network Interface + Reorder Buffer (SRAM)

Case Study: is physical implementation feasible?

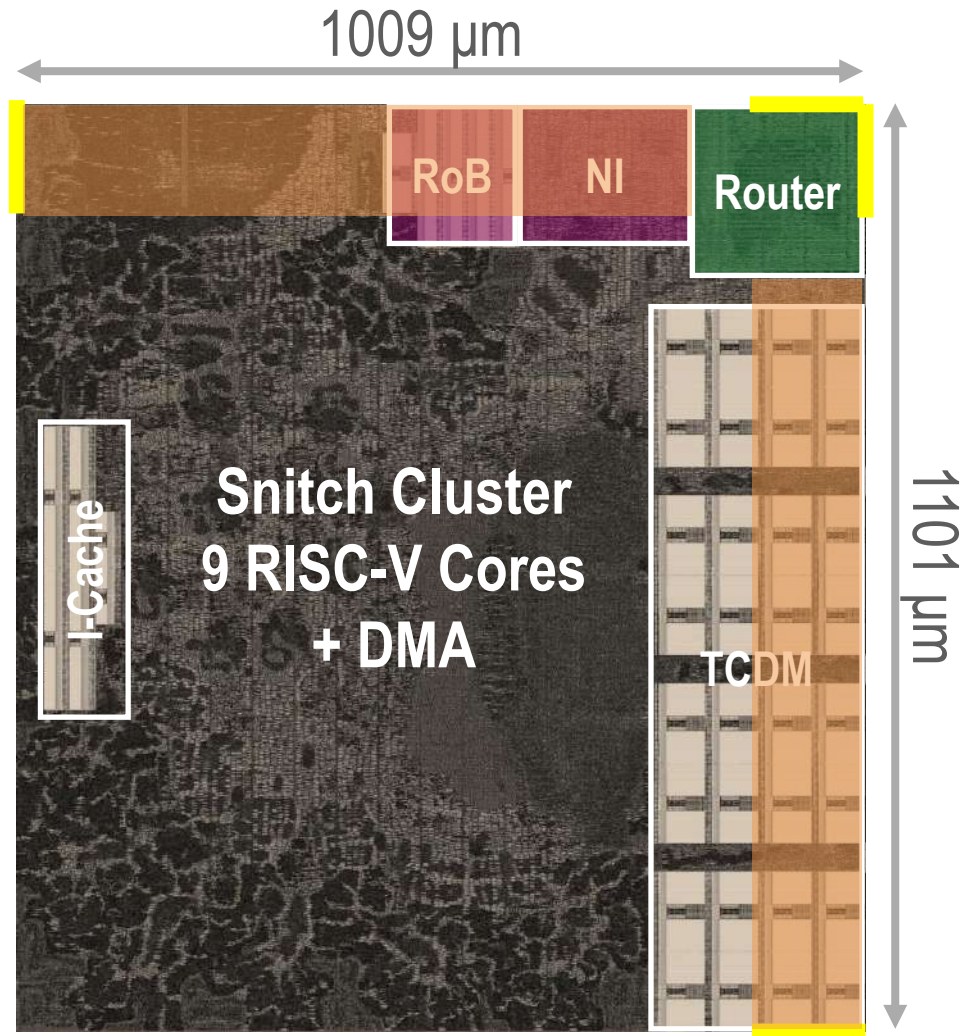


- Hard Router Macro
- Flattened Network Interface + Reorder Buffer (SRAM)

Only **10%** area overhead with an **integrated NoC**

- Tiled design methodology
 - N/E/S/W ports
 - Tiles can be directly abutted
 - Mesh of compute tiles

How to route 1000s of wires



- **Routing Channels**

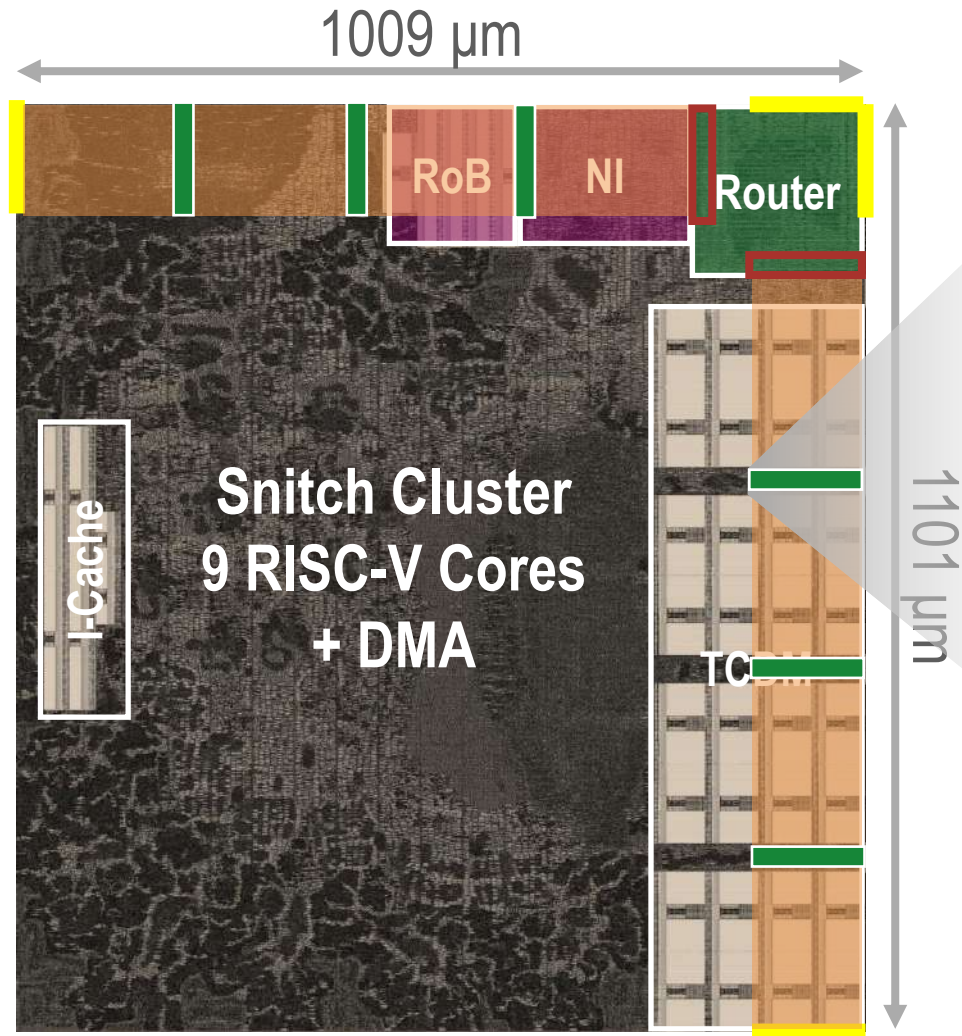
- #wires: ~1600
- Distance: 800-900 μm

- **Methodology**

- Use upper layers
 - Global routing
 - Not blocked by SRAMS
- 4 metal layers suffice with min. pitch

Area-efficient routing over
memory and logic

How many cuts do we need to meet timing?



Router

SRAM Macros

ong...

- R

S...

- ..

- ..

Buffer Island

- B

e...

- ..

SRAM Macros

rs

- Manual placement of Buffer Islands

No cuts needed to meet timing!

Low-Latency & High-Bandwidth



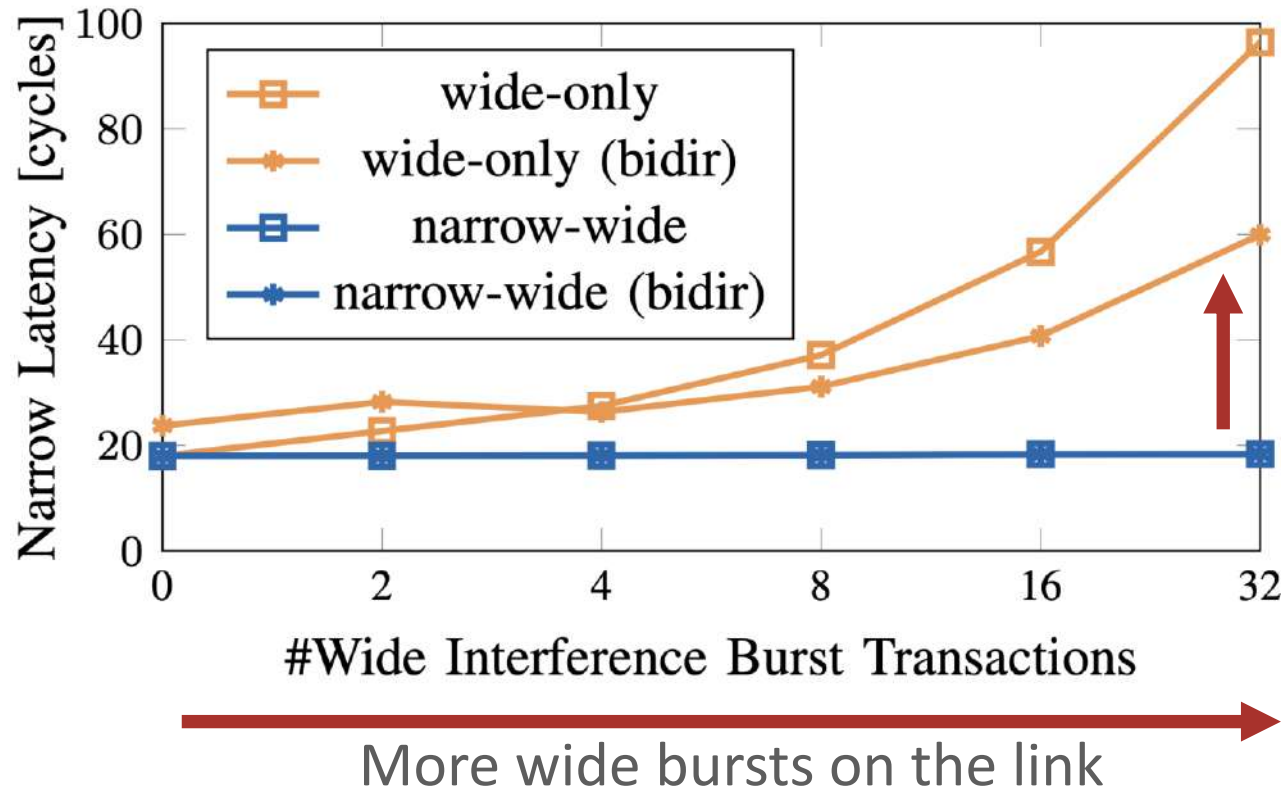
Latency

- **18 cycles** access latency to neighboring tile
 - 50% is cluster-internal latency
 - Round-trip time
- **2-cycle** hop latency
 - Even for large distances!

Bandwidth

- High Bandwidth thanks to **wide** links
 - **629Gpbs** per link
- ... at a modest **frequency**
 - **1.23 GHz** (TT, 0.8V, 25°)

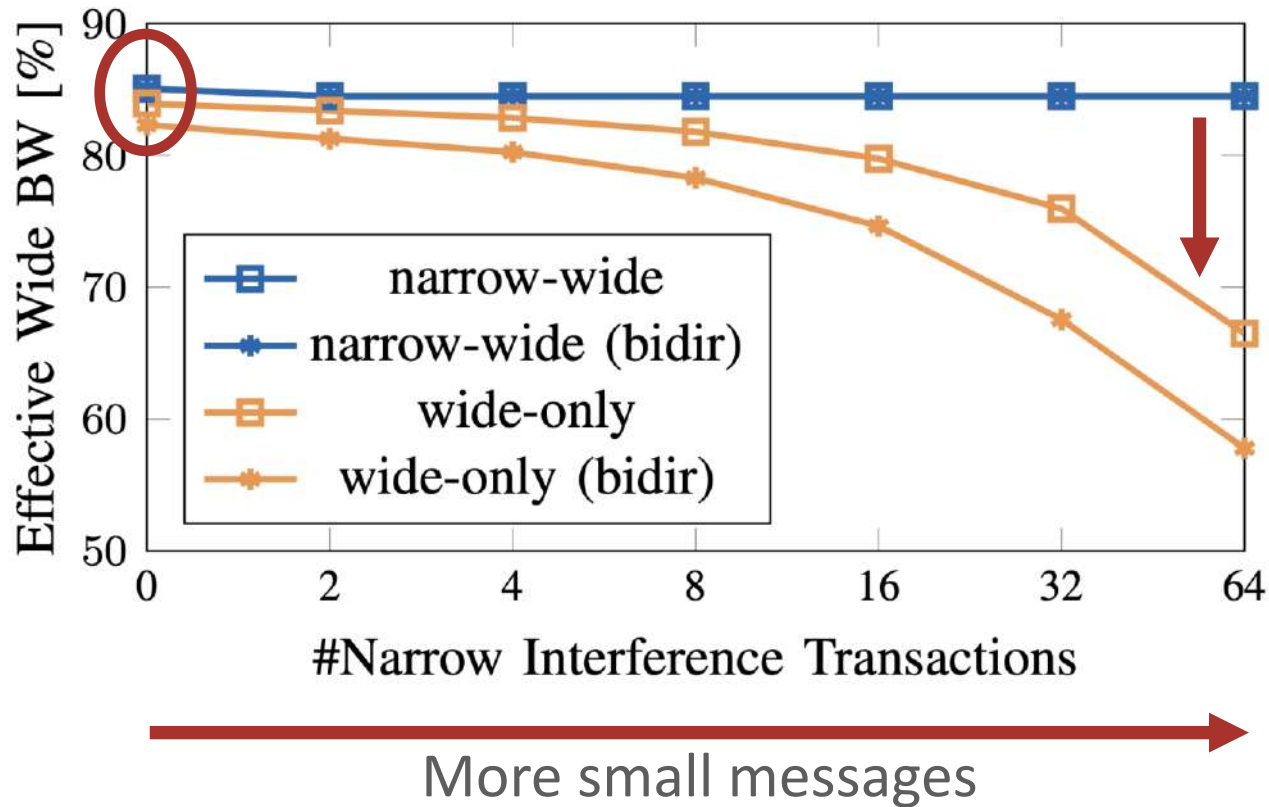
Separating channels by traffic: Latency



- **Separate vs. shared channel**
 - Wide burst transaction occupy the link for many cycles
 - Latency-sensitive messages are stalled for a longer time
 - Degradation of latency

Latency-sensitive communication benefits from separate **physical channels**

Separating channels by traffic: Link Utilization



- >80% link utilization for zero load scenario
- **Separate vs. shared channels**
 - More congestion on the link
 - Small messages not using the effective link width
 - Degradation of link utilization

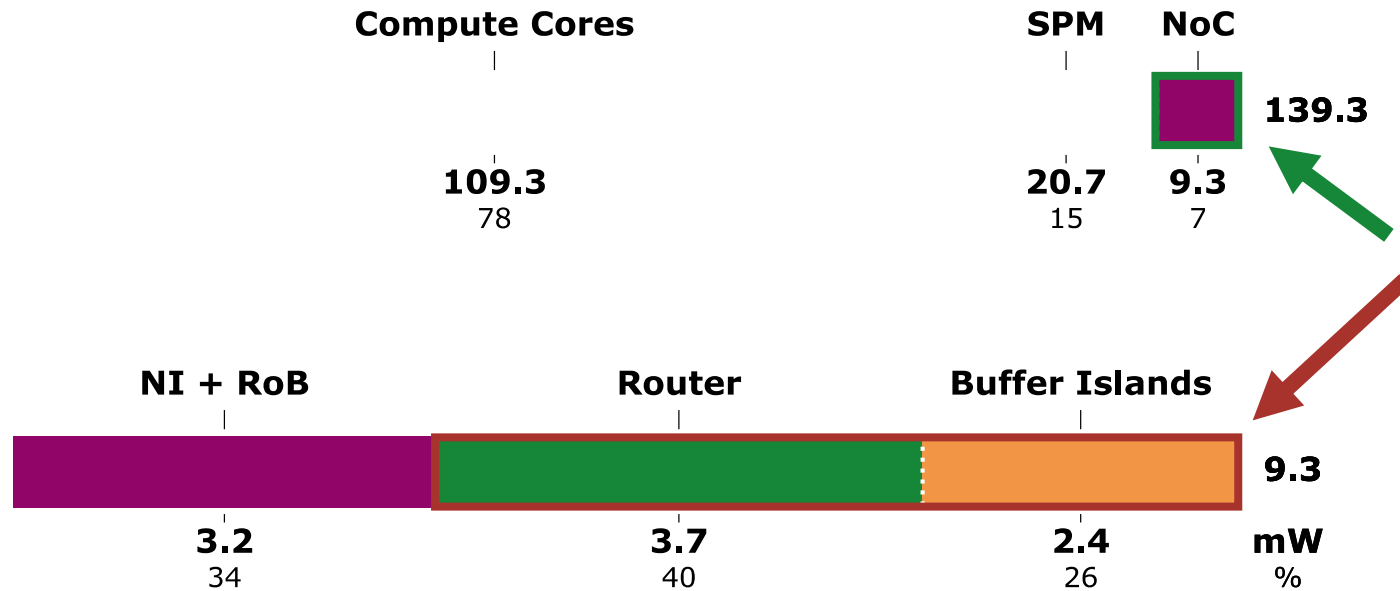
Separate **physical channels** free up the link for **high-bandwidth** transfers

Energy-efficiency is key for data-movement



Power consumption during a 1KB tile-to-tile DMA transfer

GF12nm
1.23 GHz
TT/0.8V/25°C
post-layout



- The NoC consumes **only 7%** of the power
- Data movement is very cheap with only **0.19 pJ/byte/hop**

The NoC will **not** become the energy **bottleneck** which is key for **scalability** to large systems

Conclusion



We present a **fully AXI4-compatible NoC**

- Capable of handling the **memory wall** problem
- **Open-source** end-to-end solution
- Bursts, Multiple outstanding transactions, Atomics + **Reordering**

We showed how **next-gen NoCs** should look like

- **Wide links** for high bandwidth **629Gbps/link @1.23GHz**
- Separate physical channels for **latency-sensitive** traffic
- Physical implementation of case study in GF12nm

Where Can I Get It?



FlooNoC

- Full **AXI4+ATOPs** Support
- **Wide** physical channels
- **629Gbps/link @1.23GHz**
- Only **10%** area and **7%** power overhead in a Snitch Cluster



github.com/pulp-platform/FlooNoC



arxiv.org/abs/2305.08562



How AXI addresses the memory wall problem



The good

- Open protocol
- Industry-standard, widely used
- Configurable data width
- Burst transfers
- Multiple Outstanding Transactions

AXI enables **High-Bandwidth**
and **Latency-Tolerance**

The bad

- Introduced a while ago, 2003 (AXI3) resp. 2010 (AXI4)
- Not designed as a NoC protocol
- Ordering requirements on transactions based on ID
- Transactions are tracked in every switch

AXI doesn't scale to large
systems, large protocol
overhead

About topologies and routing algorithms



Routing Algorithms

- Source-based routing
- Static & deterministic routing
- Some already implemented
 - Dimension-ordered, XY-Routing
 - Table-based Routing

Topologies

- Routers are very parametrizable
 - #ports
 - Virtual & physical channels
- Arbitrary topologies are supported
 - Mesh
 - Ring
 - ...

Easily extendable

- Designed with flexibility in mind
- Separate Routing Algorithm Module
- DIY routing algorithm
 - Dynamic routing
 - Congestion-based routing
 - etc.